

Audix ROS 2 Architecture Summary

This document turns the Copilot analysis into a practical map for this repo and this robot. It focuses on the two files that matter most right now:

- `current files/pi_master.py`: the working Raspberry Pi brain and hardware manager.
- `current files/esp_correct_pid_pi.ino`: the working ESP32 mecanum motor controller.

The goal is not to rewrite everything at once. The goal is to keep what already works, move the Pi side into ROS 2 cleanly, and understand what each ROS node/topic/service is supposed to replace.

Short Answer

The architecture should be:

```
Raspberry Pi, ROS 2 side
mission_manager_node
  subscribes: /audix/ir/state, /audix/odom, later /audix/vision/*
  calls:      /audix/move, /audix/esp/stop, later lift/buzzer services

esp_uart_bridge_node
  talks UART JSON to ESP32
  publishes: /audix/esp/telemetry, /audix/odom
  services:  /audix/move, /audix/twist, /audix/esp/stop, reset/init services

ir_gpio_node
  reads Pi GPIO IR sensors
  publishes: /audix/ir/state

gpio_outputs_node
  controls Pi buzzer, stepper/lift, traffic LEDs
  services/actions: buzzer on/off, lift up/down, set LEDs

later:
  camera_node, vision_node, tf/robot_description, robot_localization, Nav2 adapters

ESP32 side
esp_correct_pid_pi.ino
  keeps wheel PID, encoders, IMU yaw, mecanum kinematics, local odometry,
  move execution, stop/watchdog behavior, and UART command handling.
```

Main rule:

```
ESP32 closes the fast control loops.
Raspberry Pi/ROS decides what should happen next.
```

Do not move wheel PID to ROS. Do not start with micro-ROS. The current UART JSON bridge is the right first migration step.

What Copilot Got Right

Copilot's useful points:

- The robot should stay split into two layers: ESP32 low-level controller, Pi high-level ROS computer.
- `esp_correct_pid_pi.ino` already has the correct low-level responsibilities: motors, encoders, IMU, mecanum kinematics, PID, odometry, and command execution.
- `pi_master.py` is useful as the reference behavior, but it is too monolithic for ROS.

- The existing `ros2_ws` is already pointed in the right direction: a Pi ROS 2 bridge that speaks the same UART protocol to the ESP.
- micro-ROS is a later option, not the first thing to implement.
- TF matters if you want RViz, robot models, camera frames, and later Nav2 to understand where things are.

What Copilot made more overwhelming than necessary:

- It mixed immediate needs with future concepts like Nav2, full localization stacks, and behavior trees.
- It listed many packages and nodes that are valid long term, but not all are needed for the first working ROS version.
- It talked about `/cmd_vel` as an important future convention, but your current firmware works around discrete `MOVE` and `TWIST` commands. For now, services are enough.
- It mentioned sensor fusion broadly. Your ESP currently handles encoder odometry and IMU yaw for the base. Pi-side fusion can wait unless the current odometry becomes a blocker.

Current System, In Plain Terms

`pi_master.py`

`pi_master.py` currently acts like a whole robot application in one file.

It does these jobs:

Current responsibility	Where in <code>pi_master.py</code>	ROS version
UART JSON link to ESP	<code>EspPiControllink</code>	<code>esp_uart_bridge_node</code>
Send <code>MOVE</code> , wait for <code>ack/done</code>	<code>send_position_move</code> , <code>execute_position_step</code> , <code>wait_for_move_done_or_ir</code>	<code>/audix/move</code> service client in mission node
Read ESP telemetry	reader thread and <code>latest_telemetry</code>	<code>/audix/esp/telemetry</code> , <code>/audix/odom</code>
Read six IR sensors on Pi GPIO	<code>GpioIrBank</code>	<code>ir_gpio_node</code> publishing <code>/audix/ir/state</code>
Stop movement if IR triggers	<code>wait_for_move_done_or_ir</code> , <code>stop_active_move</code>	mission/safety node subscribes to IR and calls <code>/audix/esp/stop</code>
Track robot pose/progress	<code>PoseAccumulator</code> , <code>MissionMemory</code>	mostly replaced by <code>/audix/odom</code> ; mission node may keep small mission-local progress state
Obstacle behavior/state machine	<code>SimpleCardinalRealBrain</code>	<code>mission_manager_node</code>
Convert behavior command to small move steps	<code>command_to_position_step</code>	mission node calls <code>/audix/move</code>
Buzzer	<code>GpioBuzzer</code>	<code>gpio_outputs_node</code> service
Stepper/lift	<code>GpioStepperLift</code>	<code>gpio_outputs_node</code> service or action
Traffic LEDs	GPIO constants and output handling	<code>gpio_outputs_node</code>
CLI tools and run modes	argument parsing and run functions	ROS launch files, service calls, small debug scripts
Logs	<code>RunLogger</code>	ROS logs plus optional mission log node or rosbag

Important idea: `pi_master.py` should become the reference for behavior, not a file we blindly copy into ROS unchanged.

`esp_correct_pid_pi.ino`

esp_correct_pid_pi.ino is already in the correct role. It should stay responsible for low-level control.

It currently does:

ESP responsibility	Why it stays on ESP
Encoder reading	Needs fast, direct hardware timing
Wheel RPM calculation	Close to encoder timing
Per-wheel PI control	Should not depend on Linux/ROS scheduling
Mecanum inverse kinematics	Converts body motion to wheel RPMs
Position move execution	Uses local odometry and heading feedback
IMU yaw and gyro handling	Needed for heading hold
Local odometry	Comes directly from encoders and IMU
MOVE, TWIST, TURN, STOP, reset/init commands	Already proven UART contract
ack, done, telemetry JSON	Already matches Pi workflow

The Pi should ask for motion. The ESP should execute motion.

Existing ROS Workspace

The repo already has a useful first stage:

```
ros2_ws/  
  src/audix_interfaces/  
    msg/EspTelemetry.msg  
    msg/IrState.msg  
    srv/Move.srv  
    srv/TwistCommand.srv  
    srv/RawCommand.srv  
  
  src/audix_robot/  
    audix_robot/esp_uart_bridge_node.py  
    launch/audix_bridge.launch.py
```

The existing bridge node currently:

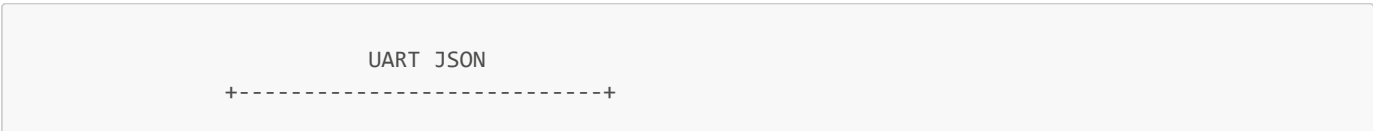
- opens the serial port to the ESP.
- sends the same text commands that pi_master.py sends.
- publishes ESP telemetry as typed ROS messages.
- publishes odometry as nav_msgs/msg/Odometry.
- reads IR GPIO itself and publishes IrState.
- exposes ROS services for move, twist, stop, ping, reset, and raw commands.

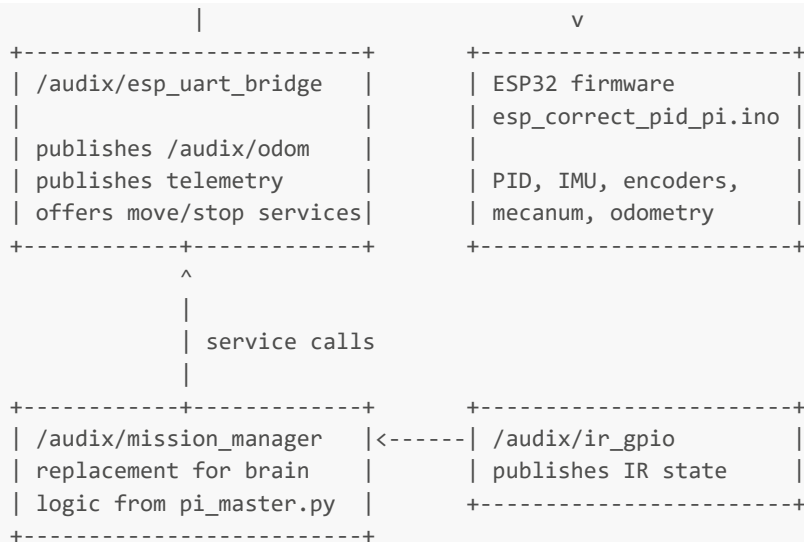
This is a good start, but it should not become the whole robot. Treat it as the hardware bridge.

Recommended Node Graph

Stage 1: Minimal ROS Version

This is the first target because it mirrors what already works.





In the current workspace, IR is inside `esp_uart_bridge_node.py`. That is acceptable temporarily. The cleaner architecture is to split IR into its own node once the bridge is stable.

Stage 2: Add Pi GPIO Outputs

```

/audix/mission_manager
  calls /audix/gpio/set_buzzer
  calls /audix/gpio/set_traffic_light
  calls /audix/lift/move_steps or /audix/lift/move action

/audix/gpio_outputs
  owns buzzer GPIO
  owns stepper/lift GPIO
  owns traffic LEDs
  
```

This replaces the Pi-side `GpioBuzzer`, `GpioStepperLift`, and traffic LED handling from `pi_master.py`.

Stage 3: Add Camera And Vision

```

/camera_node
  publishes /audix/camera/image_raw
  publishes /audix/camera/camera_info

/audix/vision_node
  subscribes /audix/camera/image_raw
  publishes /audix/vision/detections
  publishes /audix/vision/debug_image, optional

/audix/mission_manager
  subscribes /audix/vision/detections
  
```

Do this after the base and mission logic are clean in ROS.

Stage 4: Add TF, Robot Description, And RViz Friendliness

```

/audix/esp_uart_bridge
  publishes /audix/odom
  should also publish tf: odom -> base_link
  
```

```

/robot_state_publisher
  publishes fixed robot transforms from URDF
  base_link -> camera_link
  base_link -> sensor/lift frames

```

This is where ROS starts feeling like a real robot instead of just service calls.

Nodes You Need

1. `esp_uart_bridge_node`

Status: already exists in `ros2_ws/src/audix_robot/audix_robot/esp_uart_bridge_node.py`.

Purpose:

- Convert ROS services/topics into the ESP UART JSON protocol.
- Convert ESP JSON telemetry into ROS messages.
- Keep ESP firmware unchanged for now.

Publishes:

Topic	Message	Meaning
<code>/audix/esp/telemetry</code>	<code>audix_interfaces/msg/EspTelemetry</code>	Typed version of ESP telemetry: yaw, pose, RPM, PWM, encoder counts, move status
<code>/audix/esp/telemetry_raw</code>	<code>std_msgs/msg/String</code>	Raw JSON line from ESP, useful for debugging
<code>/audix/odom</code>	<code>nav_msgs/msg/Odometry</code>	ROS odometry from ESP forward/strafe/yaw
<code>/audix/ir/state</code>	<code>audix_interfaces/msg/IrState</code>	Current bridge publishes this, but it should later move to <code>ir_gpio_node</code>

Services:

Service	Type	ESP command	Meaning
<code>/audix/esp/ping</code>	<code>std_srvs/srv/Trigger</code>	PING	Check ESP is alive
<code>/audix/esp/init_imu</code>	<code>std_srvs/srv/Trigger</code>	INIT_IMU	Calibrate/zero IMU yaw if available
<code>/audix/esp/reset_odom</code>	<code>std_srvs/srv/Trigger</code>	RESET_ODOM	Reset ESP bridge odometry
<code>/audix/esp/reset_encoders</code>	<code>std_srvs/srv/Trigger</code>	RESET_ENC	Reset encoder measurements and odometry
<code>/audix/esp/stop</code>	<code>std_srvs/srv/Trigger</code>	STOP	Immediate base stop
<code>/audix/move</code>	<code>audix_interfaces/srv/Move</code>	MOVE	Move a distance at an angle while holding heading
<code>/audix/twist</code>	<code>audix_interfaces/srv/TwistCommand</code>	TWIST	Short manual RPM command
<code>/audix/esp/raw_command</code>	<code>audix_interfaces/srv/RawCommand</code>	any raw line	Debug escape hatch

Subscribers:

- None needed in the current bridge.
- Later it can subscribe to `/cmd_vel` if you want continuous ROS velocity control, but that is not required for the first migration.

Why you need it:

- It replaces `EspPiControlLink` from `pi_master.py`.
- It lets the mission logic talk in ROS instead of manually managing serial, queues, JSON, and timeouts.

What to improve later:

- Add TF broadcasting for `odom` -> `base_link`.
- Move IR GPIO out into a separate node.
- Possibly add a `/cmd_vel` subscriber or a separate `cmd_vel_adapter_node`.

2. `ir_gpio_node`

Status: recommended next split. Current bridge contains temporary IR GPIO logic.

Purpose:

- Read the six IR sensors connected to Raspberry Pi GPIO.
- Publish a clean ROS message whenever sensors are polled.

Publishes:

Topic	Message	Meaning
<code>/audix/ir/state</code>	<code>audix_interfaces/msg/IrState</code>	Boolean state of front-left, front, front-right, right, back, and left IR sensors

Subscribers:

- None.

Parameters:

Parameter	Meaning
<code>ir_logic</code>	baseline, active-low, or active-high
<code>ir_poll_period_s</code>	How often to read sensors
<code>mock_ir</code>	Run without real GPIO
pin parameters later	Useful if wiring changes

Why you need it:

- In `pi_master.py`, IR sensing is mixed with mission execution.
- In the current ROS bridge, IR sensing is mixed with ESP UART.
- A separate IR node makes the sensor data reusable by mission logic, safety logic, and debugging tools.

3. `mission_manager_node`

Status: not created yet. This is the most important new node.

Purpose:

- Replace the behavior/state-machine parts of `pi_master.py`.
- Decide the next move based on odometry, IR state, and later vision.
- Call `/audix/move` and `/audix/esp/stop`.

Publishes:

Topic	Message	Meaning
/audix/mission/state	std_msgs/msg/String or custom later	Current state such as MOVE_TO_GOAL, SHIFT_OUT, ADVANCE_CLEAR, RETURN_TO_PATH, DONE
/audix/mission/event	std_msgs/msg/String or custom later	Debug/event stream for decisions

Subscribes:

Topic	Message	Why
/audix/odom	nav_msgs/msg/Odometry	Track progress and cross-track error
/audix/ir/state	audix_interfaces/msg/IrState	Detect front/side obstacles
/audix/esp/telemetry	audix_interfaces/msg/EspTelemetry	Optional extra move status/debug
/audix/vision/detections	future custom message	Later: vision-based behavior

Service clients:

Service	Why
/audix/move	Execute one movement segment
/audix/esp/stop	Stop when IR/safety condition happens
/audix/esp/reset_odom	Reset pose at mission start if needed
/audix/gpio/*	Later: buzzer, LEDs, lift

Why you need it:

- This is where SimpleCardinalRealBrain, MissionMemory, command_to_position_step, execute_segment, and the obstacle avoidance routines belong.
- It keeps the robot's high-level decisions separate from hardware drivers.

Important design choice:

- Do not make this node manually read serial.
- Do not make this node directly drive GPIO IR pins.
- It should consume ROS topics and call ROS services.

4. gpio_outputs_node

Status: recommended after mission manager starts working.

Purpose:

- Own the Pi output devices from pi_master.py: buzzer, stepper/lift, and traffic LEDs.

Services:

Service	Suggested type	Meaning
/audix/gpio/set_buzzer	std_srvs/srv/SetBool	Buzzer on/off
/audix/gpio/set_traffic_light	custom service	Set red/yellow/green
/audix/lift/move_steps	custom service	Move stepper fixed number of steps

Possible action:

Action	Meaning
/audix/lift/move	Longer lift movement with feedback and cancellation

Why services vs actions:

- Buzzer and LEDs are quick on/off commands, so services are fine.
- A lift/stepper movement can take time and may need cancellation, so an action is better later.

5. camera_node

Status: later, when base ROS migration is stable.

Purpose:

- Publish camera frames into ROS.

Publishes:

Topic	Message	Meaning
/audix/camera/image_raw	sensor_msgs/msg/Image	Camera image
/audix/camera/camera_info	sensor_msgs/msg/CameraInfo	Calibration/info

Why you need it:

- Vision should receive images from a standard ROS image topic, not directly own all camera hardware if avoidable.

6. vision_node

Status: later.

Purpose:

- Run model/inference/classification.
- Publish detections for the mission node.

Publishes:

Topic	Message	Meaning
/audix/vision/detections	custom message later	Detected objects/targets/classes
/audix/vision/debug_image	sensor_msgs/msg/Image	Optional annotated image

Subscribes:

Topic	Message
/audix/camera/image_raw	sensor_msgs/msg/Image

7. tf_broadcaster / TF inside bridge

Status: not implemented yet, but important.

Purpose:

- Tell ROS how coordinate frames relate to each other over time.

Minimum dynamic transform:


```
odom -> base_link
```

This transform should come from the same pose used to publish `/audix/odom`.

Minimum static transforms later:

```
base_link -> camera_link
base_link -> ir_front_link, optional
base_link -> lift_link, optional
```

Why you need TF:

- `/audix/odom` says "the robot pose is x/y/yaw".
- TF says "the robot body frame is here relative to odom, and the camera/sensors are here relative to the body".
- RViz, robot_state_publisher, camera visualization, and Nav2 expect TF.

You do not need full Nav2 to need TF. TF is the common coordinate language of ROS.

Messages You Have Now

`audix_interfaces/msg/EspTelemetry`

Purpose:

- Typed ROS version of the ESP telemetry JSON.

Contains:

- IMU status and yaw.
- Forward/strafe pose in centimeters.
- Local move progress and remaining distance.
- Move phase, angle, distance, heading target/error.
- Four wheel RPMs, target RPMs, PWM values.
- Raw and signed encoder counts.
- `raw_json` for debugging.

Why you need it:

- It preserves the useful low-level data from the ESP without forcing every ROS node to parse JSON.
- It is useful for debugging tuning, motor behavior, encoder signs, and move status.

What should use it:

- Debug tools.
- Diagnostics.
- Mission node only if it needs extra move details beyond `/audix/odom`.

`audix_interfaces/msg/IrState`

Purpose:

- Report the six IR sensors as booleans.

Fields:

```
front_left
front
```

```
front_right
right
back
left
active[]
```

Why you need it:

- It replaces direct calls to `ir_bank.read()` in `pi_master.py`.
- Any node can subscribe and know what sensors are blocked.

nav_msgs/msg/Odometry

Purpose:

- Standard ROS pose/twist estimate.

In this robot:

- Published from ESP telemetry by the bridge.
- `x` should mean forward position in the ROS odom frame.
- `y` should mean lateral/strafe position.
- yaw comes from ESP IMU/odometry heading.

Why you need it:

- Mission logic should not depend on raw ESP JSON for position.
- RViz and future ROS tools expect odometry in this format.

Current caution:

- The old `pi_master.py` uses its own sign convention where physical forward maps to negative local X in `PoseAccumulator`.
- The ROS bridge uses standard ROS x-forward/y-left-ish odometry, with parameters `odom_forward_sign` and `odom_strafe_sign`.
- When testing in RViz, verify forward and left/right directions and fix signs in parameters instead of changing ESP firmware first.

Services You Have Now

/audix/move

Type: `audix_interfaces/srv/Move`

Request:

```
angle_deg
distance_m
heading_deg
timeout_s
wait_for_done
```

Response:

```
ok
result
message
forward_cm
strafe_cm
```

```
heading_deg
raw_json
```

How it maps to existing behavior:

- Replaces `execute_position_step(...)` and `link.send_position_move(...)` from `pi_master.py`.
- Sends ESP command:

```
MOVE angle=<deg> dist=<cm> heading=<deg> timeout=<ms> seq=<n>
```

When to use:

- Main mission movement.
- Obstacle avoidance segments.
- "Move 30 cm forward", "strafe right 20 cm", "return to path".

Why service is okay for now:

- The ESP already sends `ack` and then `done`.
- Your motion primitives are segmented moves, not continuous planner velocity yet.

Later improvement:

- A long move with progress/cancel semantics is more naturally an action.

`/audix/twist`

Type: `audix_interfaces/srv/TwistCommand`

Request:

```
forward_rpm
strafe_rpm
turn_rpm
timeout_s
```

How it maps to ESP:

```
TWIST forward=<rpm> strafe=<rpm> turn=<rpm> timeout=<ms>
```

When to use:

- Manual jog.
- Teleop experiments.
- Short velocity bursts.

Not the same as ROS `/cmd_vel` yet:

- `/cmd_vel` usually uses meters/sec and radians/sec in `geometry_msgs/msg/Twist`.
- This service uses RPM-like body commands because that is what the firmware accepts.

`/audix/esp/stop`

Type: `std_srvs/srv/Trigger`

How it maps:

STOP

When to use:

- Mission stop.
- IR-triggered stop.
- Emergency stop.
- Shutdown cleanup.

Reset/init services

Service	Use
<code>/audix/esp/init_imu</code>	Calibrate/zero yaw before a run
<code>/audix/esp/reset_odom</code>	Reset odometry before mission
<code>/audix/esp/reset_encoders</code>	Deeper reset if encoder state needs clearing
<code>/audix/esp/ping</code>	Confirm ESP connection
<code>/audix/esp/raw_command</code>	Debug commands like <code>STATUS</code>

Topics You Should Have

Immediate topics

Topic	Type	Publisher	Subscribers
<code>/audix/odom</code>	<code>nav_msgs/msg/Odometry</code>	<code>esp_uart_bridge_node</code>	<code>mission_manager_node</code> , RViz
<code>/audix/esp/telemetry</code>	<code>audix_interfaces/msg/EspTelemetry</code>	<code>esp_uart_bridge_node</code>	diagnostics, optional mission
<code>/audix/esp/telemetry_raw</code>	<code>std_msgs/msg/String</code>	<code>esp_uart_bridge_node</code>	debugging
<code>/audix/ir/state</code>	<code>audix_interfaces/msg/IrState</code>	current bridge, later <code>ir_gpio_node</code>	mission, safety, debug
<code>/audix/mission/state</code>	<code>std_msgs/msg/String</code> or custom later	<code>mission_manager_node</code>	CLI/RViz/debug

Later topics

Topic	Type	Why
<code>/tf</code>	<code>tf2_msgs/msg/TFMessage</code>	Required for frame relationships
<code>/audix/camera/image_raw</code>	<code>sensor_msgs/msg/Image</code>	Camera
<code>/audix/camera/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	Camera calibration
<code>/audix/vision/detections</code>	custom later	Vision results
<code>/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>	Standard ROS velocity command convention
<code>/audix/emergency_stop</code>	<code>std_msgs/msg/Bool</code> or service/action design	Safety

Actions You May Need Later

You do not need actions for the first step, but these are the places where actions fit better than services:

Action	Why action instead of service
<code>/audix/move_base_segment</code>	Long-running move with feedback and cancellation
<code>/audix/run_mission</code>	Full mission can report state/progress and be cancelled
<code>/audix/lift/move</code>	Stepper movement may take time and should be cancellable
<code>/audix/search_target</code>	Vision/search behavior can take time and report feedback

For now, keep `/audix/move` as a service because it matches the working ESP `ack/done` protocol.

Where `/cmd_vel` Fits

`/cmd_vel` is standard ROS. It usually carries:

```
geometry_msgs/msg/Twist
linear.x    forward m/s
linear.y    strafe m/s, useful for mecanum
angular.z   yaw rad/s
```

Your firmware currently accepts:

```
TWIST forward=<rpm> strafe=<rpm> turn=<rpm> timeout=<ms>
MOVE  angle=<deg> dist=<cm> heading=<deg>
```

So do not force `/cmd_vel` into the first migration. Later, add one of these:

1. `cmd_vel_adapter_node`: subscribes `/cmd_vel`, converts m/s and rad/s into ESP `TWIST` commands.
2. Add `/cmd_vel` subscriber directly inside `esp_uart_bridge_node`.

The adapter node is cleaner because it keeps the bridge focused on transport.

TF Explained For This Robot

TF is not a sensor. It is ROS's live coordinate-frame system.

For Audix, the important frames are:

Frame	Meaning
<code>odom</code>	Local world frame. Starts wherever the robot begins. It can drift.
<code>base_link</code>	Robot body center. This moves as the robot moves.
<code>camera_link</code>	Camera's physical frame on the robot.
<code>base_footprint</code>	Optional flat ground projection of the robot base.

Minimum useful TF tree:

```
odom
  -> base_link
    -> camera_link
```

Why Copilot mentioned TF:

- `/audix/odom` gives pose data, but many ROS tools do not only read odometry messages.

- RViz wants to know where `base_link` is relative to `odom`.
- If you add a camera, ROS needs to know where the camera is mounted relative to the robot.
- If you later use Nav2, TF becomes mandatory.

What to implement first:

- In `esp_uart_bridge_node`, publish `odom` -> `base_link` using the same x/y/yaw as `/aux/odom`.

What can wait:

- Full URDF.
- All sensor frames.
- Nav2 transform complexity.

Micro-ROS Decision

Do not start with micro-ROS.

Reason:

- Your ESP firmware already works as a low-level controller.
- It already has a tested UART command contract.
- Rewriting it into a native micro-ROS node would mix a firmware rewrite with the ROS migration.

Correct path:

```
Now:
  ESP firmware unchanged
  Pi runs ROS 2
  Pi bridge speaks UART JSON

Later, optional:
  Replace UART JSON firmware with micro-ROS firmware
  ESP publishes/subscribes as a native ROS participant
```

If micro-ROS happens later, the ESP should still keep:

- wheel PID
- encoder reading
- mecanum kinematics
- IMU heading handling
- immediate stop/watchdog logic

micro-ROS would replace the communication layer, not the control responsibility.

What To Implement First

Step 1: Verify the existing ROS bridge

Goal:

- Confirm the ROS bridge can do what `pi_master.py` already does at the hardware interface level.

Commands to verify on the Pi:

```
cd ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
```

```
source install/setup.bash
ros2 launch audix_robot audix_bridge.launch.py port:=/dev/ttyAMA0
```

Then test:

```
ros2 service call /audix/esp/ping std_srvs/srv/Trigger {}
ros2 topic echo /audix/esp/telemetry
ros2 topic echo /audix/odom
ros2 topic echo /audix/ir/state
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 0.0, distance_m: 0.20,
heading_deg: 0.0, timeout_s: 8.0, wait_for_done: true}"
ros2 service call /audix/esp/stop std_srvs/srv/Trigger {}
```

Expected result:

- ESP responds to ping.
- Telemetry streams.
- Odom changes when the robot moves.
- IR state changes when sensors are blocked.
- Move service completes with `ok: true` and result `completed`, or stops safely if interrupted.

Step 2: Add TF to the bridge

Goal:

- Publish `odom` -> `base_link`.

Why second:

- It is small but unlocks RViz and makes ROS coordinate behavior more standard.

Step 3: Split IR into `ir_gpio_node`

Goal:

- Move `GpioIrBank` out of `esp_uart_bridge_node.py`.

Why:

- UART bridge should not own Pi IR hardware.
- Mission node should not care whether IR came from GPIO, mock mode, or another sensor source.

Step 4: Create `mission_manager_node`

Goal:

- Port the useful logic from `SimpleCardinalRealBrain` and mission execution functions.

First version should:

- subscribe to `/audix/odom`.
- subscribe to `/audix/ir/state`.
- call `/audix/move`.
- call `/audix/esp/stop`.
- publish `/audix/mission/state`.

Do not add camera or Nav2 yet.

Step 5: Add GPIO outputs node

Goal:

- Move buzzer, stepper/lift, and traffic LEDs out of `pi_master.py`.

First version:

- `set_buzzer` service.
- `set_traffic_light` service.
- `lift_move_steps` service.

Later:

- Lift action with feedback/cancel.

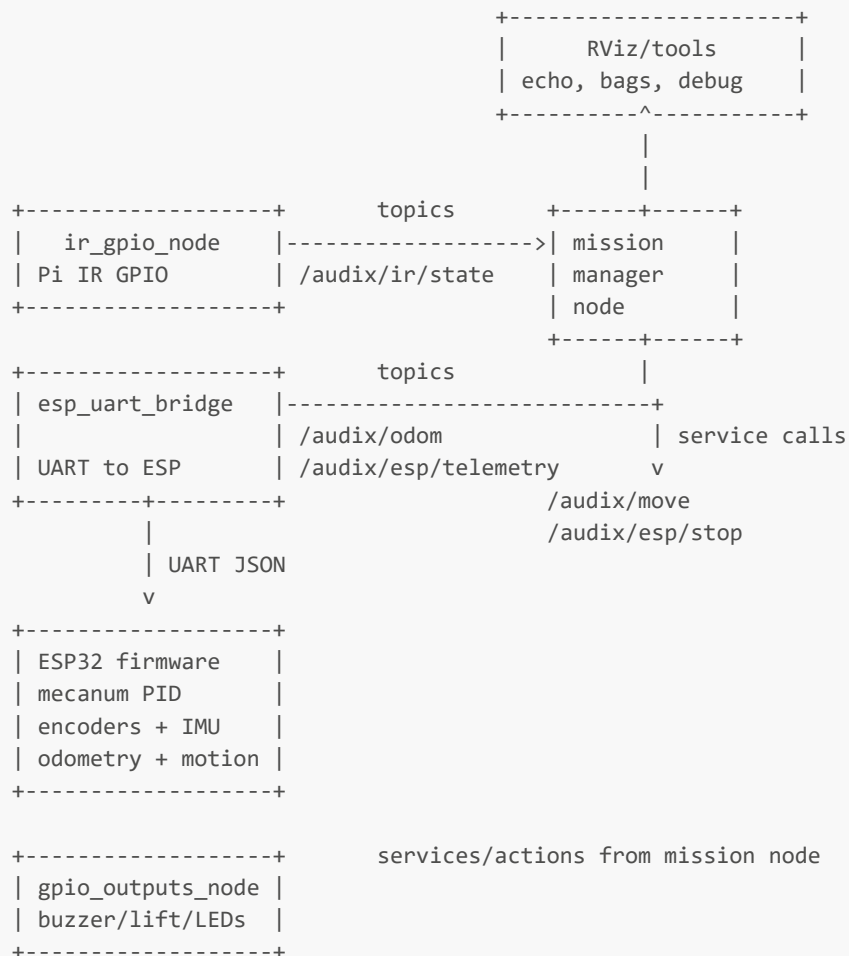
Step 6: Add camera and vision

Only after the base ROS architecture works.

What Not To Do Yet

- Do not rewrite ESP firmware into micro-ROS yet.
- Do not move PID loops to the Pi.
- Do not start with Nav2.
- Do not design a large custom message system before the basic bridge and mission node are stable.
- Do not make the bridge node become the new giant `pi_master.py`.
- Do not duplicate IR reading in both the bridge and another node long term.

Clean Target Architecture



Later:


```
camera_node -> vision_node -> mission_manager_node  
tf/URDF -> RViz/Nav2 readiness
```

Practical Translation Of Existing Behavior

Current non-ROS behavior:

```
pi_master.py reads IR  
pi_master.py decides next segment  
pi_master.py sends MOVE over serial  
ESP executes movement  
ESP sends ack/done/telemetry  
pi_master.py updates mission state
```

ROS behavior:

```
ir_gpio_node publishes /audix/ir/state  
esp_uart_bridge_node publishes /audix/odom and /audix/esp/telemetry  
mission_manager_node decides next segment  
mission_manager_node calls /audix/move  
esp_uart_bridge_node sends MOVE over serial  
ESP executes movement  
esp_uart_bridge_node receives ack/done/telemetry  
mission_manager_node updates mission state from service result and topics
```

That is the same robot behavior, but split into understandable ROS parts.

Final Recommendation

Start by making the current ROS bridge trustworthy, then port the Pi brain into a `mission_manager_node`. That is the real migration. Everything else, including micro-ROS, Nav2, full localization, and vision, should come after the basic base-control and mission loop work in ROS.

The first useful milestone is:

```
With the ESP firmware unchanged, run ROS 2 on the Pi and command the robot  
through /audix/move while watching /audix/odom and /audix/ir/state.
```

Once that works, `pi_master.py` becomes a reference document instead of the thing driving the robot.